LOW-Level Design (LLD)

RAG-Based Customer Support Assistant using LangGraph

Prepared by:Thippani Srinija

Intern Id:IN226024402

Table of Contents

1 Module -Level Design 1

1.1 **Document Processing Module 1**
1.2 **Chunking Module 1**
1.3 **Embedding Module 1**
1.4 **Vector Storage Module 1**
1.5 **Retrieval Module 2**
1.6 **Query Processing Module 2**
1.7 **Graph Execution Module 2**
1.8 **HITL Module 2**

2 **Data Structures 3**

2.1 **Document Representation 3**
2.2 **Chunk Format 3**
2.3 **Embedding Structure 3**
2.4 **Query-Response Schema 3**
2.5 **State Object for Graph 3**

3 **Workflow Design 4**

3.1 **Nodes 4**
3.2 **Edges 4**
3.3 **State Flow 4**

4 **Conditional Routing Logic 5**

4.1 **Answer Generation Criteria 5**
4.2 **Escalation Criteria 5**
4.3 **Low Confidence Conditions 5**
4.4 **Missing Context Handling 5**

5 **HITL Design 6**

5.1 **Trigger Conditions 6**
5.2 **Process Flow 6**
5.3 **Integration 6**

6 **API/Interface Design 7**

6.1 **Input Format 7**
6.2 **Output Format 7**
6.3 **Interaction Flow 7**

7 **Error Handling 8**

7.1 **Missing Data 8**
7.2 **No Relevant Chunks 8**
7.3 **System Failure 8**
7.4 **Invalid Input 8**

OBJECTIVE:

To define the internal working, modules, data structures, and execution logic of the RAG-based customer support system in detail.

1. MODULE-LEVEL DESIGN

1.1 Document Processing Module

This module is responsible for ingesting and preparing the input PDF document.

- Uses PyPDFLoader to read PDF files
- Extracts text content from all pages
- Handles multiple pages and formats
- Cleans unwanted characters or formatting issues

Output: Raw structured text ready for chunking

1.2 Chunking Module

This module divides large text into manageable segments.

- Uses RecursiveCharacterTextSplitter
- Maintains:
    - chunk_size = 500
    - chunk_overlap = 100
- Ensures context continuity between chunks
- Improves retrieval accuracy

Output: List of meaningful text chunks

1.3 Embedding Module

This module converts text into vector representations.

- Uses sentence-transformers (all-MiniLM-L6-v2)
- Converts both:
    - Document chunks
    - User queries
- Captures semantic meaning instead of exact keywords

Benefit: Enables semantic similarity search

1.4 Vector Storage Module

Stores embeddings in a vector database.

- Uses ChromaDB
- Stores:
    - Chunk text
    - Corresponding embeddings

- Supports fast nearest-neighbor search
- Persistent storage (./chroma_db)

Acts as the knowledge base

1.5 Retrieval Module

Responsible for fetching relevant data.

- Converts user query → embedding
- Performs similarity search
- Retrieves top-k relevant chunks (k=3)
- Uses cosine similarity

Output: Most relevant chunks

1.6 Query Processing Module

Processes retrieved chunks to generate final response.

- Extracts relevant answer from chunks
- Filters unnecessary content
- Matches query with correct question-answer pair
- Ensures concise and meaningful response

Avoids noisy or irrelevant output

1.7 Graph Execution Module

Controls workflow using LangGraph.

- Defines nodes:
    - process_node
    - output_node
    - human_node
- Defines edges (flow logic)
- Maintains state transitions

Works as execution engine

1.8 HITL Module (Human-in-the-Loop)

Handles cases where system cannot answer confidently.

- Triggered when:
    - No relevant chunk found
    - Low confidence response
- Prompts human for input
- Returns human-generated answer

Improves reliability

2. DATA STRUCTURES

2.1 Document Representation

Stores extracted PDF text:

Code:

```
{
  "pages": ["text_page_1", "text_page_2"]
}
```

2.2 Chunk Format

Code:

```
{
  "chunk_id": "1",
  "content": "Customers can return products within 7 days..."
}
```

2.3 Embedding Structure

Code:

```
{
  "vector": [0.12, -0.45, 0.78, ...],
  "dimension": 384
}
```

2.4 Query-Response Schema

Code:

```
{
  "query": "How can I track my order?",
  "answer": "Use the tracking link sent via email or SMS."
}
```

2.5 State Object for Graph

Code:

```
{
  "query": "user input",
  "answer": "generated answer",
  "confidence": "high/low"
}
```

3. WORKFLOW DESIGN (LangGraph)

3.1 Nodes

i)Process Node

- Handles query processing
- Calls retriever
- Generates answer

ii)Output Node

- Displays final answer to user

iii)Human Node

- Handles escalation
- Accepts human input

3.2 Edges (Flow Logic)

process → output
process → human (if low confidence)
human → output

3.3 State Flow

- State object passes between nodes
- Maintains:
  - Query
  - Answer
  - Confidence

4. CONDITIONAL ROUTING LOGIC

4.1 Answer Generation Criteria

- Relevant chunk found
- Query matches stored knowledge
- Output is meaningful

4.2 Escalation Criteria

- No relevant chunks retrieved
- Output is empty or unclear
- Query not presents in PDF

4.3 Low Confidence Conditions

- Weak similarity score
- Incomplete answer
- No keyword match

4.4 Missing Context Handling

- System detects absence of relevant data
- Returns fallback or triggers HITL

5. HITL DESIGN

5.1 Trigger Conditions

- Low confidence
- No answer found

5.2 Process Flow

System → detects low confidence
↓
Prompt human
↓
Human inputs answer
↓
Return to user

5.3 Integration

- Human response overrides system response
- Ensures accuracy and trust

6. API / INTERFACE DESIGN

6.1 Input Format

Code:

```
{
  "query": "What is return policy?"
}
```

6.2 Output Format

Code:

```
{
  "answer": "Customers can return products within 7 days..."
}
```

6.3 Interaction Flow

User → Input → Retriever → LangGraph → Output

7. ERROR HANDLING

7.1 Missing Data

- Return: “No data available in knowledge base”

7.2 No Relevant Chunks

- Trigger fallback or HITL

7.3 System Failure

- Return: “Something went wrong. Please try again.”

7.4 Invalid Input

- Ask user to re-enter query